

Natural Language Processing - LLM Module

Artificial Intelligence

Jacopo Parretti

I Semester 2025-2026

Indice

I	Foundations of Neural Networks	3
1	Introduction to Neural Networks	3
1.1	The Concept of a Neuron	3
1.2	Activation Functions	3
1.3	Multi-Layer Neural Networks	4
2	Learning in Neural Networks	5
2.1	The Learning Problem	5
2.2	Optimization Methods	5
2.3	From Numbers to Text	5
3	Introduction to Transformers	6
3.1	Why Transformers?	6
3.2	Transfer Learning and Foundation Models	7
3.3	Self-Supervised Learning	7
3.4	The Scaling Hypothesis	7
4	The Attention Mechanism	8
4.1	Motivation: Context-Dependent Representations	8
4.2	Attention vs. Standard Neural Networks	8
5	Transformer Processing: Mathematical Framework	9
5.1	Notation and Terminology	9
5.2	Transformer Layer: The Main Building Block	10
6	Attention Coefficients	11
6.1	The Goal of Attention	11
6.2	Linear Combination Case	11
7	Self-Attention Mechanism	11
7.1	How to Compute Attention Coefficients	11
7.2	Self-Attention in Transformers	12
7.3	Summary: Self-Attention Transformation	12
8	Learnable Parameters in Self-Attention	13
8.1	The Problem with Simple Self-Attention	13
8.2	Solution: Query, Key, and Value Transformations	13
8.3	Modified Self-Attention Formula	14
8.4	General Formula and Dimensions	14
8.5	Key Distinction: Data-Dependent Attention	15

Parte I

Foundations of Neural Networks

1 Introduction to Neural Networks

1.1 The Concept of a Neuron

Neural networks are computational models inspired by biological neural systems. At their core, they consist of interconnected units called **neurons** (or **nodes**).

Definition: Artificial Neuron

An **artificial neuron** is a mathematical function that:

1. Receives multiple inputs $(x_1, x_2, \dots, x_n)$
2. Computes a weighted sum of these inputs plus a bias term
3. Applies an activation function to produce an output

1.1.1 Mathematical Formulation

The computation performed by a single neuron can be expressed as:

$$z = \sum_{i=1}^n w_i x_i + b = w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b$$

where:

- x_i are the input values
- w_i are the **weights** associated with each input
- b is the **bias** term
- z is the weighted sum (also called the **pre-activation**)

The final output of the neuron is obtained by applying an **activation function** σ to the weighted sum:

$$y = \sigma(z) = \sigma \left(\sum_{i=1}^n w_i x_i + b \right)$$

1.2 Activation Functions

Activation functions introduce non-linearity into the network, enabling it to learn complex patterns. Common activation functions include:

Common Activation Functions

1. Sigmoid:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Outputs values in the range $(0, 1)$. Often used for binary classification.

2. ReLU (Rectified Linear Unit):

$$\text{ReLU}(z) = \max(0, z)$$

Outputs z if $z > 0$, otherwise 0. Most commonly used in modern deep learning.

3. Tanh (Hyperbolic Tangent):

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Outputs values in the range $(-1, 1)$. Centers data around zero.

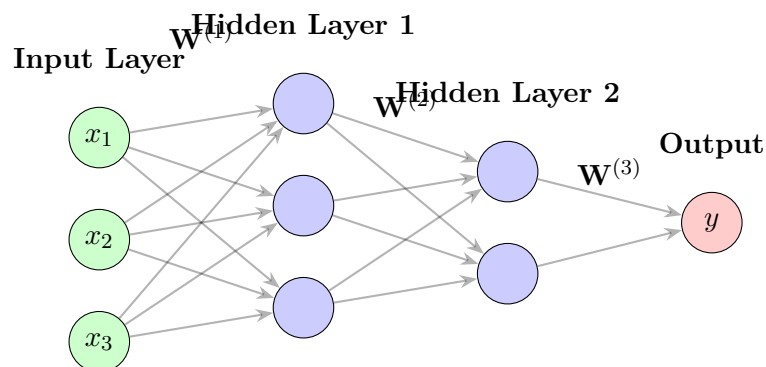
1.3 Multi-Layer Neural Networks

The power of neural networks comes from connecting multiple neurons in **layers** to form deep architectures.

Definizione 1.1 (Feedforward Neural Network). A **feedforward neural network** (also called a **multilayer perceptron**) consists of:

- An **input layer**: receives the input features
- One or more **hidden layers**: perform intermediate computations
- An **output layer**: produces the final prediction

1.3.1 Architecture Visualization



1.3.2 Weight Matrices

Each layer in a neural network is associated with a **weight matrix** that defines the connections between neurons.

- $\mathbf{W}^{(1)}$: Weight matrix connecting the input layer to the first hidden layer
- $\mathbf{W}^{(2)}$: Weight matrix connecting the first hidden layer to the second hidden layer
- $\mathbf{W}^{(3)}$: Weight matrix connecting the second hidden layer to the output layer

For a layer with n input neurons and m output neurons, the weight matrix $\mathbf{W}$ has dimensions $m \times n$.

2 Learning in Neural Networks

2.1 The Learning Problem

Objective: Learning the Weights

The fundamental goal of training a neural network is to **learn the optimal values of the weight matrices** $\mathbf{W}^{(1)}, \mathbf{W}^{(2)}, \dots, \mathbf{W}^{(L)}$ that minimize the prediction error on the training data.

This is analogous to learning the coefficients in a regression model:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$$

where $\beta_0, \beta_1, \dots, \beta_n$ are the parameters to be learned.

2.2 Optimization Methods

Several optimization algorithms are used to learn neural network weights:

2.2.1 Gradient Descent

Definizione 2.1 (Gradient Descent). **Gradient descent** is an iterative optimization algorithm that updates the weights in the direction opposite to the gradient of the loss function:

$$\mathbf{W} \leftarrow \mathbf{W} - \eta \nabla_{\mathbf{W}} \mathcal{L}$$

where:

- η is the **learning rate** (step size)
- $\nabla_{\mathbf{W}} \mathcal{L}$ is the gradient of the loss function with respect to the weights

2.2.2 ADAM Optimizer

ADAM (Adaptive Moment Estimation)

ADAM is an advanced optimization algorithm that combines ideas from momentum and adaptive learning rates. It is one of the most popular optimizers in modern deep learning.

Key features:

- Adapts learning rates for each parameter individually
- Uses moving averages of gradients (momentum)
- Computationally efficient and requires little memory
- Works well in practice with default hyperparameters

2.3 From Numbers to Text

Osservazione. While neural networks fundamentally operate on **numerical data**, they can be adapted to work with various data types, including:

- **Images:** Represented as matrices of pixel values
- **Audio:** Represented as waveforms or spectrograms
- **Text:** Represented through embeddings (numerical vectors)

This versatility enables neural networks to perform tasks such as **text generation**, **machine translation**, and **image captioning**.

3 Introduction to Transformers

Key Concept: Transformers

All of the preceding discussion serves as foundation for understanding a specific and revolutionary type of neural network architecture: **Transformers**.

Transformers are the backbone of modern large language models (LLMs) such as GPT, BERT, and many others. They have revolutionized natural language processing and are now being applied to various domains beyond text.

3.1 Why Transformers?

Transformers represent one of the most important developments in deep learning. First introduced in the field of natural language processing, they have greatly surpassed previous architectures such as **Recurrent Neural Networks (RNNs)** in both performance and efficiency.

3.1.1 Core Concept: Attention Mechanism

The Attention Mechanism

Transformers are based on a processing concept called **attention**. The attention mechanism allows the model to focus on different parts of the input when producing each part of the output, weighing the importance of different input elements dynamically.

Why "Transformers"?

The name comes from their ability to **transform input vectors** from one representation space to another, creating richer internal representations suitable for downstream tasks.

3.1.2 Input Flexibility

Transformers can process various types of inputs:

- **Unstructured sets of vectors:** No inherent ordering required
- **Ordered sequences:** Natural for text and time series
- **General representations:** Broad applicability across domains

3.2 Transfer Learning and Foundation Models

Transfer Learning with Transformers

Transfer learning is particularly effective with Transformers. The typical workflow involves:

1. **Pre-training:** Train a large model on a massive corpus of data
2. **Fine-tuning:** Adapt the pre-trained model to specific downstream tasks with smaller, task-specific datasets

This approach has led to the concept of **foundation models**—large-scale models trained on broad data that serve as a foundation for various applications.

3.3 Self-Supervised Learning

Osservazione. Transformers can be trained in a **self-supervised** manner using unlabeled data. This is crucial because:

- Labeled data is expensive and time-consuming to obtain
- Unlabeled text data is abundant (e.g., web pages, books, articles)
- Self-supervised objectives (e.g., predicting masked words) provide strong learning signals

3.4 The Scaling Hypothesis

Scaling Hypothesis

The **scaling hypothesis** states that by increasing both:

- The **scale of the model** (number of learnable parameters)
- The **size of the training dataset**

significant improvements in model performance can be achieved.

This hypothesis has been empirically validated across numerous experiments, leading to increasingly large models.

3.4.1 Massive Scale and Parallel Processing

Transformers are particularly well-suited to **massively parallel processing**, which enables:

- Training of very large models with parameters on the order of **trillions** (10^{12})
- Reasonable training times despite model size (using distributed computing and specialized hardware like GPUs and TPUs)
- Efficient inference for real-world applications

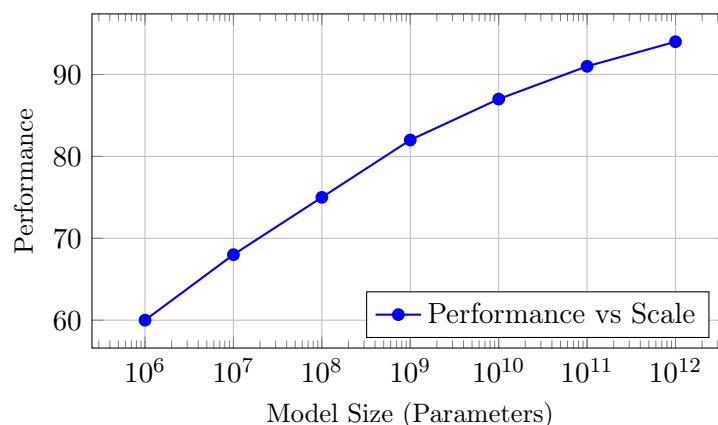


Illustration: As model size increases (log scale), performance improves following the scaling hypothesis.

4 The Attention Mechanism

4.1 Motivation: Context-Dependent Representations

Esempio 4.1 (Word Ambiguity). Consider the following two sentences containing the word "bank":

- *I swam across the river to get to the other **bank***
- *I walked across the road to get cash from the **bank***

To determine the exact interpretation of the word *bank*, an artificial neural network should **rely more heavily on** (i.e., **attend to**) specific words from the rest of the sentence (**context**).

In the first sentence, "bank" should attend more to words like "river" and "swam" (indicating a riverbank). In the second sentence, "bank" should attend more to words like "cash" and "road" (indicating a financial institution).

4.2 Attention vs. Standard Neural Networks

Key Difference

Standard neural networks: When the network is trained, weights are **fixed**.

Attention mechanism: Uses **weighting factors** whose values **depend on the specific input data**.

Osservazione. A Transformer can be seen as a **rich form of embedding** in which a given vector (e.g., representing a word) is mapped to a location that **depends on the other vectors** (i.e., words) of the sentence.

Example: The embedding of the word "bank" will be:

- Close to "water" in the first sentence
- Close to "money" in the second sentence

This context-dependent representation is what makes Transformers so powerful for natural language understanding.

5 Transformer Processing: Mathematical Framework

5.1 Notation and Terminology

Definizione 5.1 (Input Representation). The input to a Transformer consists of:

- **Set of vectors:** $\{x_n\}$ of dimensionality D where $n = 1, \dots, N$
- Each vector is called a **token**

5.1.1 What is a Token?

A **token** is a fundamental unit of input data. Examples include:

- A **word** in a sentence
- A **patch** in an image
- An **amino acid** in a protein sequence
- A **number or set of numbers** in a time series

5.1.2 Features

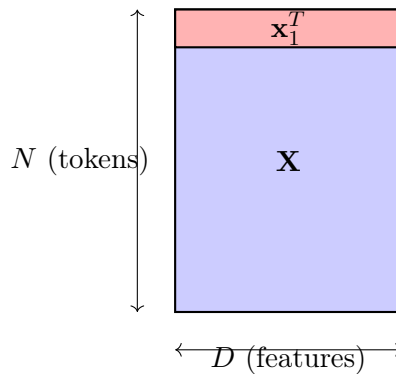
The elements x_{ni} of a token are called **features**, where $i = 1, \dots, D$.

5.1.3 Data Matrix

Definizione 5.2 (Data Matrix). The **data matrix** (set of tokens) is denoted by $\mathbf{X}$ with dimension $N \times D$:

- Each **row** is a token vector $\mathbf{x}_n^T$
- N is the number of tokens
- D is the number of features (dimensionality)

A **training set** contains many sets of tokens (i.e., many matrices).



5.2 Transformer Layer: The Main Building Block

Transformer Layer

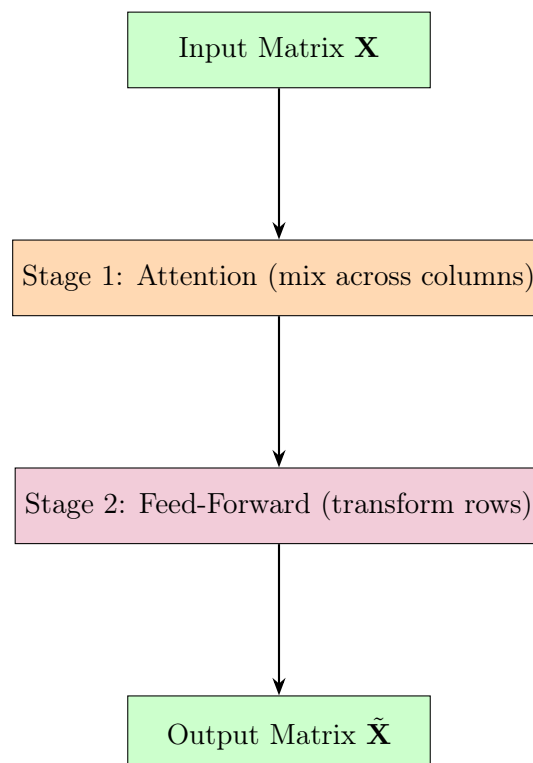
A **Transformer layer** is the fundamental building block of a Transformer that:

- Takes a **data matrix $\mathbf{X}$** as input
- Returns a **transformed matrix $\tilde{\mathbf{X}}$** of the same dimension as output

$$\tilde{\mathbf{X}} = \text{TransformerLayer}[\mathbf{X}]$$

5.2.1 Key Properties

1. **Multiple layers:** Multiple Transformer layers can be applied in succession to learn **rich internal representations**
2. **Learnable weights:** Each Transformer layer contains weights that can be learned by **gradient descent** using an appropriate **cost function**
3. **Two-stage processing:** Each Transformer layer performs two stages:
 - (a) **Stage 1 (Attention):** Mixes features from different tokens across the **columns** of $\mathbf{X}$ (attention mechanism)
 - (b) **Stage 2 (Feed-forward):** Transforms the values of each token acting on the **rows** of $\mathbf{X}$ independently



6 Attention Coefficients

6.1 The Goal of Attention

Attention Objective

Given a set of tokens in an embedding space: $\mathbf{x}_1, \dots, \mathbf{x}_N$

Goal: Map them into another set $\mathbf{y}_1, \dots, \mathbf{y}_N$ with richer semantic structure.

Key principle: The value of a generic output token $\mathbf{y}_n$ should **depend on all input tokens**.

Attention: The dependence should be **stronger** for inputs $\mathbf{x}_m$ particularly important for determining the new representation of $\mathbf{y}_n$.

6.2 Linear Combination Case

In the simplest case, we can express the output as a **linear combination** of the input tokens:

$$\mathbf{y}_n = \sum_{m=1}^N a_{nm} \mathbf{x}_m$$

where:

- a_{nm} are called **attention weights**
- Constraint: $a_{nm} \geq 0$ and $\sum_{m=1}^N a_{nm} = 1$ (partition of unity to avoid compensation)

Definizione 6.1 (Attention Weights). The coefficients a_{nm} represent how much the n -th output token should "attend to" the m -th input token. They form a probability distribution over the input tokens for each output position.

Interpretation:

- If a_{nm} is large, token $\mathbf{x}_m$ has a strong influence on $\mathbf{y}_n$
- If a_{nm} is small, token $\mathbf{x}_m$ has little influence on $\mathbf{y}_n$
- The sum constraint ensures that the total attention is normalized

7 Self-Attention Mechanism

7.1 How to Compute Attention Coefficients

7.1.1 Example: Movie Selection (Hard Attention)

To understand how attention coefficients work, consider a simplified example of movie selection based on user preferences.

Esempio 7.1 (Movie Recommendation with Hard Attention). Given:

- **Preference (query):** A user's preference vector representing their interests
- **Movies database:** A collection of movies, each with:
 - **Keys:** Feature vectors describing movie properties
 - **Values:** The actual movie content/representation

The system computes a **similarity score** $Q^T K$ between the user's query and each movie's key. In **hard attention**, only the movie with the highest similarity score is selected.

7.2 Self-Attention in Transformers

In Transformers, the attention mechanism is more sophisticated than the hard attention example above. Here's how it works:

Self-Attention: Key Concepts

- Each input token $\mathbf{x}_n$ serves **both** as a **key** and a **value** (i.e., the token itself summarizes its features)
- The token $\mathbf{x}_m$ is used as a **query** for the output $\mathbf{y}_m$ (then $\mathbf{x}_m$ is compared to all the key tokens)
- The dot product $\mathbf{x}_n^T \mathbf{x}_m$ is used to compute the **query-key similarity**, which determines how much the token represented by $\mathbf{x}_n$ should **attend to** the token represented by $\mathbf{x}_m$ (i.e., how $\mathbf{x}_n$ is important to generate $\mathbf{y}_m$)

7.2.1 Computing Attention Weights with Softmax

Constraints are imposed using the **softmax function**:

$$a_{nm} = \frac{\exp(\mathbf{x}_n^T \mathbf{x}_m)}{\sum_{m'=1}^N \exp(\mathbf{x}_n^T \mathbf{x}_{m'})}$$

This formula ensures:

- All attention weights are positive: $a_{nm} > 0$
- They sum to one: $\sum_{m=1}^N a_{nm} = 1$
- Higher dot products yield higher attention weights (exponential scaling)

7.3 Summary: Self-Attention Transformation

Self-Attention Formula

Each **input vector** $\mathbf{x}_n$ is transformed to a corresponding **output vector** $\mathbf{y}_n$ by taking a **linear combination** of input vectors:

$$\mathbf{y}_n = \sum_{m=1}^N a_{nm} \mathbf{x}_m$$

where the weight a_{nm} applied to vector $\mathbf{x}_m$ is computed by the softmax of the dot-product between the **query** $\mathbf{x}_n$ and the **key** $\mathbf{x}_m$.

7.3.1 Matrix Notation

In matrix form, the self-attention operation can be written as:

$$\mathbf{Y} = \text{Softmax}[\mathbf{X}\mathbf{X}^T]\mathbf{X}$$

where:

- $\mathbf{X}$ is the $N \times D$ input matrix (N tokens, D features)
- $\mathbf{X}^T$ is the $D \times N$ transpose

- $\mathbf{XX}^T$ is an $N \times N$ similarity matrix
- Softmax is applied row-wise to normalize each row
- $\mathbf{Y}$ is the $N \times D$ output matrix

Interpretation of $\mathbf{XX}^T$:

- This is an $N \times N$ matrix where element (n, m) contains $\mathbf{x}_n^T \mathbf{x}_m$
- Each row represents how similar token n is to all other tokens
- After softmax, each row becomes the attention weights for that token

8 Learnable Parameters in Self-Attention

8.1 The Problem with Simple Self-Attention

Osservazione. The basic self-attention transformation $\mathbf{Y} = \text{Softmax}[\mathbf{XX}^T]\mathbf{X}$ has **no learnable parameters**.

Additionally, each feature value within a token vector $\mathbf{x}_n$ plays an **equal role** in determining the attention coefficients. This limits the expressiveness of the model.

8.2 Solution: Query, Key, and Value Transformations

Modified Feature Vectors

Define **modified feature vectors** as linear transformations of the original vectors:

$$\begin{aligned}\mathbf{Q} &= \mathbf{X}\mathbf{W}^{(q)} && \text{(Queries)} \\ \mathbf{K} &= \mathbf{X}\mathbf{W}^{(k)} && \text{(Keys)} \\ \mathbf{V} &= \mathbf{X}\mathbf{W}^{(v)} && \text{(Values)}\end{aligned}$$

where $\mathbf{W}^{(q)}$, $\mathbf{W}^{(k)}$, and $\mathbf{W}^{(v)}$ are **learnable parameter matrices**.

8.2.1 Dimensions

The weight matrices have the following dimensions:

- $\mathbf{W}^{(q)}$: $D \times D_q$ (not necessarily $D_q = D$)
- $\mathbf{W}^{(k)}$: $D \times D_k$
- $\mathbf{W}^{(v)}$: $D \times D_v$

Usually, $D_k = D_v = D$, so:

- Dimension of output = dimension of input
- **Multiple Transformer layers can be stacked** sequentially

8.2.2 Resulting Matrices

After transformation:

- $\mathbf{Q}$: $N \times D_q$ matrix of query vectors

- $\mathbf{K}$: $N \times D_k$ matrix of key vectors
- $\mathbf{V}$: $N \times D_v$ matrix of value vectors

8.3 Modified Self-Attention Formula

With the learnable transformations, the self-attention mechanism becomes:

$$\mathbf{Y} = \text{Softmax} [\mathbf{Q}\mathbf{K}^T] \mathbf{V}$$

or equivalently:

$$\mathbf{Y} = \text{Softmax} \left[\mathbf{X}\mathbf{W}^{(q)} \left(\mathbf{X}\mathbf{W}^{(k)} \right)^T \right] \mathbf{X}\mathbf{W}^{(v)}$$

Key benefits:

- The model can **learn** which features are important for queries, keys, and values
- Different features can play different roles in computing attention
- The weight matrices $\mathbf{W}^{(q)}$, $\mathbf{W}^{(k)}$, $\mathbf{W}^{(v)}$ are trained via gradient descent

8.4 General Formula and Dimensions

General Self-Attention Formula

The general formula for computing the self-attention transformation is:

$$\mathbf{Y} = \text{Softmax}[\mathbf{Q}\mathbf{K}^T] \mathbf{V}$$

where:

- $\mathbf{Q}\mathbf{K}^T$ has dimension $N \times N$ (attention weight matrix)
- $\mathbf{Y}$ has dimension $N \times D_v$ (output matrix)

8.4.1 Dimensional Analysis

Let's trace through the dimensions step by step:

1. **Input:** $\mathbf{X}$ has dimension $N \times D$

2. **Query transformation:**

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^{(q)}$$

- $\mathbf{X}$: $N \times D$
- $\mathbf{W}^{(q)}$: $D \times D$
- $\mathbf{Q}$: $N \times D$

3. **Key transformation:**

$$\mathbf{K} = \mathbf{X}\mathbf{W}^{(k)}$$

- $\mathbf{X}$: $N \times D$
- $\mathbf{W}^{(k)}$: $D \times D$
- $\mathbf{K}$: $N \times D$

4. Attention scores:

$$\mathbf{QK}^T$$

- $\mathbf{Q}$: $N \times D$
- $\mathbf{K}^T$: $D \times N$
- $\mathbf{QK}^T$: $N \times N$

5. Value transformation:

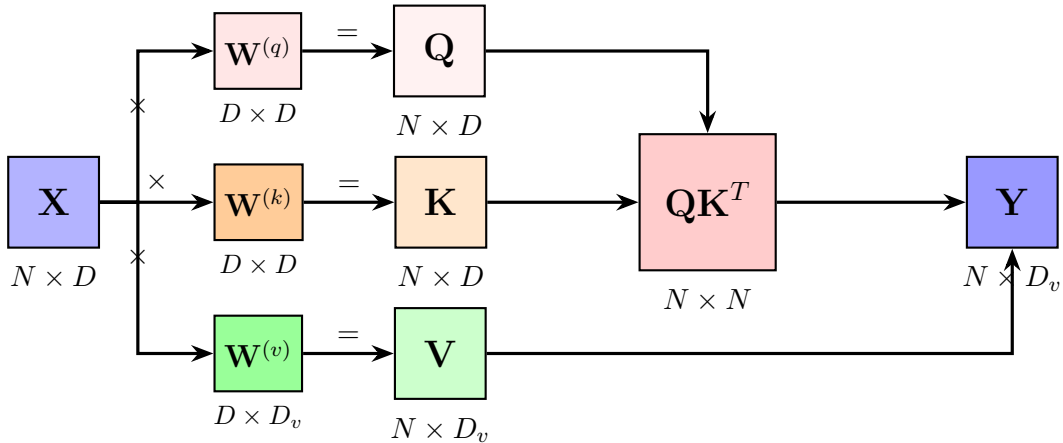
$$\mathbf{V} = \mathbf{XW}^{(v)}$$

- $\mathbf{X}$: $N \times D$
- $\mathbf{W}^{(v)}$: $D \times D_v$
- $\mathbf{V}$: $N \times D_v$

6. Final output:

$$\mathbf{Y} = \text{Softmax}[\mathbf{QK}^T]\mathbf{V}$$

- $\text{Softmax}[\mathbf{QK}^T]$: $N \times N$
- $\mathbf{V}$: $N \times D_v$
- $\mathbf{Y}$: $N \times D_v$



8.5 Key Distinction: Data-Dependent Attention

Important Notice

Standard neural networks multiply activations by **fixed weights**.
 Transformers multiply activations by **data-dependent attention coefficients**.
 This is a fundamental difference that gives Transformers their power and flexibility.

8.5.1 Example: Input-Specific Attention

Esempio 8.1 (Selective Attention). Consider two different input vectors $\mathbf{x}'$ and $\mathbf{x}''$:

- An attention coefficient could be **close to zero** for input vector $\mathbf{x}'$
 - This means the incoming signal is **ignored** in the signal path
- The same attention coefficient could be **very large** for another input vector $\mathbf{x}''$
 - This means the incoming signal is **strongly considered**

Key insight: By contrast, if a standard neural network learns to ignore an input, it does so **for all input vectors**. The network cannot selectively attend based on context.

Osservazione. This data-dependent weighting allows Transformers to:

- **Dynamically route information** based on input content
- **Context-sensitive processing:** The same token can be processed differently depending on surrounding tokens
- **Flexible feature selection:** Different features become important for different inputs
- **Adaptive computation:** The network's effective structure changes with each input

This is why Transformers excel at tasks requiring understanding of context and relationships between elements, such as natural language processing and sequence modeling.